

KLINIKOS DARBUOTOJŲ PAMAINŲ GRAFIKO SUDARYMO IR KONFLIKTŲ SPRENDIMO SISTEMA

Kursinio darbo I dalis: projektavimo dokumentas

1. Problema ir idėja

Sistema vienu sakiniu: Sistema skirta medicinos klinikos skyriaus darbuotojų pamainoms paskirstyti pagal darbuotojų prieinamumą, darbo valandų ribas, poilsio reikalavimus ir nustatytas preferencijas.

Problema ir dabartinis procesas: Grafiką skyriaus vadovas sudaro rankiniu būdu, dažniausiai naudodamas skaičiuoklę. Keičiantis darbuotojų prieinamumui ar pamainų poreikiui, reikia iš naujo tikrinti konfliktus, darbo valandų balansą ir darbuotojų pageidavimus. Dėl to procesas imlus laikui ir didėja žmoniškų klaidų tikimybė.

Nauda: Sistema automatizuos kandidatų tinkamumo tikrinimą, pamainos balo skaičiavimą ir konfliktų sprendimą. Vadovas gaus ne tik priskirtą darbuotoją, bet ir sprendimo priežastį bei balą. 12 valandų poilsio taisyklė šiame projekte laikoma klinikos nustatyta verslo taisykle, o ne bendru teisiniu teiginiu.

Naudotojai: Skyriaus vadovas įveda pamainų poreikį, darbuotojų prieinamumą ir preferencijas bei paleidžia grafiko sudarymą. Darbuotojas pateikia savo pamainų preferencijas ir peržiūri galutinį grafiką.

Prielaidos: Planuojama sudaryti savaitės grafiką; mėnesio grafikas formuojamas iš atskirų savaitių. Paminos yra fiksuotos 8 val. trukmės: dieninė D (08:00-16:00), vakarinė V (16:00-24:00), naktinė N (00:00-08:00). Kiekvieno darbuotojo savaitinis maksimalus valandų limitas, jau priskirtos savaitės valandos, paskutinės pamainos pabaiga, tos savaitės priskirtos pamainos ir preferencijos yra žinomi prieš skaičiavimą. Atostogos ir nedarbingumas įvedami prieš grafiko generavimą.

2. Apimtis

Funkcija	Ką naudotojas galės atlikti	Tipas
Pamainų priskyrimas ir konfliktų sprendimas	Pagal pateiktas pamainas ir darbuotojų duomenis sistema tikrina tinkamumą, skaičiuoja balą ir kiekvienai pamainai parenka geriausią kandidatą pagal nustatytą taisyklių seką.	Pagrindinis modulis
Darbuotojų preferencijų valdymas	Darbuotojas gali nurodyti pamainų tipus prioriteto tvarka, pvz., D pirmas, V antras.	Pagalbinė funkcija
Prieinamumo, atostogų ir nedarbingumo žymėjimas	Vadovas nurodo, kuriomis dienomis darbuotojas gali arba negali būti skiriamas į pamainą.	Pagalbinė funkcija
Grafiko peržiūra ir eksportas	Vadovas ir darbuotojai peržiūri sugeneruotą kalendorinį grafiką ir gali jį eksportuoti CSV formatu.	Pagalbinė funkcija

Į kursinio darbo apimtį neįeina: Integracija su išorinėmis darbo laiko apskaitos ar socialinio draudimo sistemomis, SMS pranešimų siuntimas, darbo užmokesčio skaičiavimas ir globaliai optimalios tvarkaraščio paieškos metodai (pvz., pilnas matematinis optimizavimas). Prototipe naudojama deterministinė kandidatų parinkimo taisyklių seka.

3. Pagrindinis modulis

Pavadinimas ir atsakomybė: Pamainų priskyrimo ir konfliktų sprendimo variklis (ShiftOptimizationEngine). Jo atsakomybė - apdoroti pamainas chronologine tvarka, atmesti kandidatų neatitiktis pagal griežtus apribojimus, apskaičiuoti tinkamumo balą ir deterministiškai pasirinkti vieną kandidatą.

Logika, kurią reikės projektuoti ir testuoti: Kandidato prieinamumo ir kitų griežtų apribojimų tikrinimas; tinkamumo balo skaičiavimas; lygiųjų sprendimas pagal apibrėžtą prioritetų seką; klaidos grąžinimas, kai tinkamo kandidato nėra.

Įvestis: Pamainų sąrašas (ID, data, tipas, pradžia, pabaiga, trukmė), darbuotojų sąrašas (ID, savaitinis limitas, jau priskirtos savaitės valandos, paskutinės pamainos pabaiga, tos savaitės priskirtos pamainos, preferencijos) ir prieinamumo informacija. Pavyzdys: S1 = 2026-10-12 D (08:00-16:00, 8 h); E1 = max_hours 40, current_hours 32, last_shift_end 2026-10-10 16:00, prefs [D, V].

Išvestis: Priskirtos pamainos ID, darbuotojo ID, apskaičiuotas balas ir sprendimo priežastis. Jei kandidatų nėra, grąžinama klaida ERR_NO_ELIGIBLE_EMPLOYEE su priežastimi.

Veikimo eiga: 1) Paminos surikiuojamos pagal pradžios laiką. 2) Kiekvienai pamainai patikrinami visi darbuotojai. 3) Netinkami kandidatai atmetami pagal T1-T4. 4) Likusiems skaičiuojamas T5 balas. 5) Didžiausią balą surinkusiam kandidatui

taikomas T6 lygiųjų sprendimas. 6) Priskyrimas išsaugomas ir atnaujinamos darbuotojo savaitės valandos bei paskutinės pamainos duomenys. 7) Jei tinkamo kandidato nėra, grąžinama klaida.

Taisyklės arba sprendimo žingsniai

- T1 - griežtas prieinamumo apribojimas: darbuotojas turi būti pažymėtas kaip galintis dirbti konkrečią pamainą ir tą dieną neturėti atostogų ar nedarbingumo. Jei nepasiekiamas, kandidatas atmetamas.
- T2 - griežtas savaitės valandų limitas: $\text{current_hours} + \text{shift_duration} \leq \text{max_hours}$. Jei sąlyga netenkinama, kandidatas atmetamas.
- T3 - griežtas minimalus poilsis: $\text{time_between}(\text{last_shift_end}, \text{shift_start}) \geq 12$ val. Jei paskutinės pamainos nėra, T3 laikoma tenkinama. 12 val. yra šio projekto verslo taisyklė.
- T4 - viena pamaina per kalendorinę dieną: darbuotojas negali turėti jau priskirtos pamainos tą pačią datą.
- T5 - tinkamumo balas: $S_{\text{total}} = S_{\text{pref}} + S_{\text{bal}}$. $S_{\text{pref}} = 50$, jei pamainos tipas yra 1-as pasirinkimas; 25, jei 2-as pasirinkimas; 0, jei pamainos tipo preferencijose nėra. $S_{\text{bal}} = 50 - (\text{current_hours} / \text{max_hours}) * 50$. Skaičiavimas taikomas tik kandidatams, praėjusiems T1-T4.
- T6 - lygiosios sprendžiamos deterministiškai: 1) mažesnis current_hours ; 2) jei valandos sutampa, mažesnis darbuotojo ID abėcėline / leksikografinė tvarka.

Scenarijai būsimiems testams

Scenarijus	Pradinės sąlygos ir konkreti įvestis	Veiksmas	Tikslus laukiamas rezultatas
Įprastas atvejis	S1: 2026-10-12 D (08:00-16:00). E1: 32/40 h, prefs [D,V]. E2: 24/40 h, prefs [N,D]. E3: 32/40 h, prefs [V,N].	Vykdyti assign(S1).	E1: $S_{\text{pref}} 50 + S_{\text{bal}} 10 = 60.0$; E2: $25 + 20 = 45.0$; E3: $0 + 10 = 10.0$. S1 priskiriama E1; weekly hours tampa 40 h.
Ribinis atvejis / konfliktas	S2: 2026-10-12 N (00:00-08:00). E1: 32/40 h, prefs [N], last_shift_end 2026-10-11 16:00. E2: 32/40 h, prefs [N], last_shift_end 2026-10-11 08:00. E3: 20/40 h, prefs [V,N], last_shift_end 2026-10-11 08:00.	Vykdyti assign(S2).	E1 atmetamas pagal T3 (8 h < 12 h). E2 balas 60.0; E3 balas 50.0. S2 priskiriama E2.
Klaida / rezultato nėra	S3: 2026-10-12 D (08:00-16:00). E1: 36/40 h. E2: 40/40 h. E3: 32/40 h, bet 2026-10-12 jau turi N pamainą, pasibaigiančią 08:00.	Vykdyti assign(S3).	E1 ir E2 atmetami pagal T2 (viršija max 40 h limitą). E3 atmetamas pagal T3 (0 h < 12 h) ir T4 (tą pačią dieną jau turi pamainą). Grąžinama ERR_NO_ELIGIBLE_EMPLOYEE.
Lygiosios	S4: 2026-10-13 D. E1 ir E2 abu turi 24/40 h ir prefs [D]. Abu atitinka T1-T3.	Vykdyti assign(S4).	Abiejų balas 70.0. T6 taikomas 1 žingsniu ir nieko nesuranda, todėl taikomas 2 žingsnis: pagal ID E1 < E2. S4 priskiriama E1.

Jei modulis naudoja AI: Netaikoma. Pagrindinio modulio sprendimai yra deterministiniai; AI nepriima pamainų priskyrimo sprendimų. Tai leidžia kiekvieną taisyklę ir rezultatą patikrinti automatizuotais testais.

4. Kokybės atributas

Pasirinktas atributas: Palaikomumas (Maintainability).

Kodėl svarbus šiai sistemai: Paminų taisyklės gali būti keičiamos ar papildomos. Pagrindinis algoritmas neturėtų būti perrašomas kiekvieną kartą, kai įvedamas naujas apribojimas.

Tikrinimo scenarijus ir sąlygos: Vėlesniame etape bus pridėta T7 taisyklė – ne daugiau kaip 2 naktinės pamainos vienam darbuotojui per planuojamą savaitę. Nauja taisyklė bus realizuota atskirame objekte, kuris atitinka IConstraintRule sąsają ir įtraukiamas į variklio taisyklių sąrašą.

Sėkmės kriterijus: T7 galima pridėti nepakeičiant ShiftOptimizationEngine pagrindinio priskyrimo algoritmo logikos; visi ankstesni T1-T5 testai išlieka sėkmingi, o naujasis testas teisingai atmeta darbuotoją, turintį 2 naktines pamainas.

Numatytas projektavimo sprendimas: Atskira IConstraintRule sąsaja ir taisyklių sąrašas. Kiekviena griežta taisyklė realizuojama atskirame modulyje; variklis jas kviečia vienoda sąsaja.

Kaip patikrinsiu vėlesniame etape: Sukursiu unit testą Maintainability_NewRule_Test, pridėsiu T7 į taisyklių sąrašą, vykdysiu visą ankstesnių testų rinkinį ir patikrinsiu, kad nauja taisyklė keičia tik numatytą kandidatų tinkamumo rezultatą.

Sprendimo kaina arba ribojimas: Atsiranda papildomos sąsajos ir atskirų taisyklių klasės, todėl pradinis kodas šiek tiek didesnis nei vienos klasės realizacija. Už tai gaunamas aiškesnis ir lengviau plečiamas taisyklių mechanizmas.

Papildomas atributas: Našumas (Performance)

Kodėl svarbus: Grafiko sudarymas turi būti pakankamai greitas, kad naudotojui nereikėtų laukti kiekvienos pamainos skaičiavimo.

Tikrinimo scenarijus ir sąlygos: 100 darbuotojų ir 150 pamainų; kiekvienai pamainai tikrinami visi potencialūs kandidatai, todėl gaunama iki 15 000 kandidatų vertinimų vienam generavimui. Testas vykdomas vietinėje dokumentuotoje aplinkoje.

Sėkmės kriterijus: 95 % viso grafiko generavimo paleidimų (P95) trunka mažiau nei 500 ms.

Numatytas projektavimo sprendimas: Darbuotojų būsenos laikomos atmintyje generavimo metu; DB užklauskos nevykdomos kandidato parinkimo vidiniame cikle. Algoritmo bazinė sudėtingumo riba šiame prototipe yra $O(S \times E)$, kur S - pamainų, E - darbuotojų skaičius.

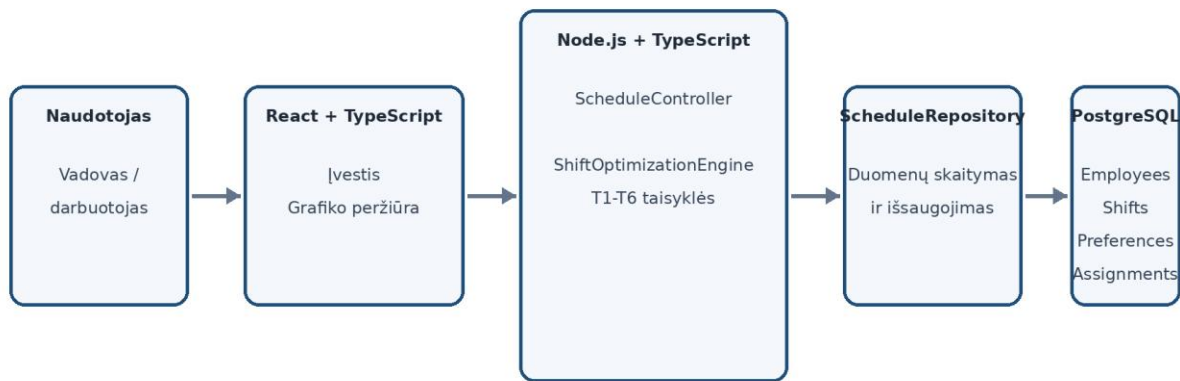
Kaip patikrinsiu vėlesniame etape: Automatizuotas našumo testas paleidžiamas bent 100 kartų su fiksuotu sintetiniu duomenų rinkiniu, fiksuojamas vykdymo laikas ir apskaičiuojamas P95.

Sprendimo kaina arba ribojimas: Daugiau būsenos laikoma RAM atmintyje, todėl reikia aiškiai apibrėžti jos gyvavimo laiką ir saugiai atnaujinti duomenis po kiekvieno priskyrimo.

5. Pradinė sistemos struktūra

Pradinė sistemos struktūra

Naudotojo įvestis → verslo logika → duomenų saugojimas



1 pav. Pradinė sistemos komponentų schema

Sistemos dalis	Atsakomybė
Kliento sąsaja (React + TypeScript)	Surinkti vadovo įvestį, parodyti darbuotojų preferencijas, generavimo būseną ir galutinį grafiką.
ScheduleController (Node.js + TypeScript)	Priimti REST užklausą, atlikti pagrindinę įvesties validaciją ir perduoti duomenis varikliui.
ShiftOptimizationEngine	Vykdyti T1-T6 verslo logiką, skaičiuoti balus, spręsti konfliktus ir grąžinti rezultatą.
ScheduleRepository	Skaityti darbuotojų, pamainų ir preferencijų duomenis bei išsaugoti priskyrimus.
PostgreSQL	Saugoti darbuotojus, pamainas, preferencijas ir priskyrimus, užtikrinant duomenų vientisumą.

Planuojamos technologijos ir pasirinkimo priežastys: Backend - Node.js ir TypeScript: viena kalba tarp sluoksnių ir griežtas tipavimas padeda sumažinti klaidų riziką verslo logikoje. HTTP API - Express: paprasta REST API prototipo realizacija. Frontend - React + TypeScript: komponentinė sąsaja ir patogus kalendorinio grafiko atvaizdavimas. Duomenų bazė - PostgreSQL: reliacinė struktūra darbuotojų, pamainų ir priskyrimų duomenims bei transakcijų palaikymas. Testai - Jest: unit testams pagal T1-T6 ir kokybės kriterijui tikrinti.

6. AI panaudojimas

AI rengiant šį dokumentą

Priemonė ir užduotis	Ką panaudojau	Ką atmečiau arba perrašiau ir kodėl	Kaip patikrinau
ChatGPT ir Google Gemini - idėjos, struktūra ir logikos peržiūra	Panaudotos idėjos apie pamainų apribojimus, balų skaičiavimą, testų struktūrą ir dokumento skyrius.	Atmesta formuluočių, kad algoritmas garantuoja „globaliai optimalų“ grafiką. Taip pat atsisakyta teisinį atitikimą	Taisyklės ir testų skaičiai perskaičiuoti pagal dokumente pateiktas formules; patikrinta, ar

PROGRAMŲ SISTEMŲ PROJEKTAVIMAS

Priemonė ir užduotis	Ką panaudojau	Ką atmečiau arba perrašiau ir kodėl	Kaip patikrinau
		garantuojančio teiginio ir palikta 12 val. taisyklė kaip projekto prielaida.	testai sutampa su T1-T6.
AI pagalba dokumento redagavimui	Panaudotas kalbos ir struktūros suvienodinimas, kad visi skyriai atitiktų šablono klausimus.	Pašalinti šablono paaiškinimai ir neapibrėžta „Node.js arba Python“ alternatyva.	Kiekvienas skyriaus teiginys sutikrintas su šablono laukais ir užduoties reikalavimais.

Planuojamas AI naudojimas kuriant sistemą

Kur ir kam naudosiu AI: AI asistentą planuojama naudoti boilerplate kodo, DTO / tipų aprašų ir unit testų ruošiniams generuoti bei kodo alternatyvoms palyginti.

Kaip tikrinsiu pasiūlymus ir sugeneruotą kodą: Kiekvieną pasiūlymą pirmiausia peržiūrėsiu rankiniu būdu, tada tikrinsiu pagal T1-T6 taisykles ir automatinius unit testus. Sugeneruotas kodas nebus laikomas teisingu vien todėl, kad jį pateikė AI.

Ar AI bus sistemos funkcionalumo dalis: Ne. AI bus naudojamas kaip kūrimo pagalbinė priemonė, bet pats pamainų priskyrimo modulis veiks pagal deterministines taisykles.

7. Tolesnių darbų planas

Darbas	Apčiuopiamas rezultatas	Planuojama darbų seka
1. Duomenų modelio ir DB projektavimas	PostgreSQL schema: employees, shifts, preferences, assignments, availability.	Sukurti lenteles, migracijas ir pradinius testinius duomenis.
2. Pagrindinio modulio realizacija	ShiftOptimizationEngine ir atskiros T1-T6 taisyklių realizacijos.	Pirmiausia įgyvendinti hard constraints, tada balų skaičiavimą ir T5 tie-breaker.
3. Unit testai	Automatiniai T1-T6 ir 4 dokumento scenarijų testai.	Jest testai; kiekvienas testas tikrina konkretų rezultatą, ne tik „success“ būseną.
4. REST API ir UI prototipas	Endpoint grafiko generavimui ir React forma.	Sujungti UI -> API -> Engine -> Repository; parodyti priskyrimų priežastis ir balus.

Būsimo prototipo veikimo scenarijus: Vadovas įveda 3 darbuotojus (E1, E2, E3) ir 5 pamainas. Viena iš jų sukelia poilsio konflikto situaciją. Sistema pamainas apdoroja chronologine tvarka, parodo kiekvienam kandidatui apskaičiuotą balą, nurodo atmestus kandidatus ir jų priežastis, o pabaigoje pateikia sugeneruotą grafiką.

Rizika arba neaiškumas	Kaip patikrinsiu arba sumažinsiu
Neužtenka darbuotojų visoms pamainoms	Naudoti aiškią ERR_NO_ELIGIBLE_EMPLOYEE būseną ir prototipe parodyti neužpildytą pamainą su atmetimo priežastimis.
Algoritmas per lėtas didesniame rinkinyje	Sukurti našumo testą su 100 darbuotojų ir 150 pamainų; viršijus P95 tikslą, pirmiausia profiluoti T1-T6 ciklus ir optimizuoti kandidatų filtravimą.